

Drift Monitoring System: Checking the Pulse of ML Models

Objective: Learn how to monitor ML systems for Data Drift vs Concept Drift and implement automated checks.

Presenter: Antigravity (AI Tutor)

Target Audience: Beginners in MLOps

Problem Statement

- **The Issue:** Models trained on static data rot in production.
- **Why:** The world changes (Inflation, User Behavior, Policy).
- **Consequence:** Model accuracy drops silently.
- **Goal:** Build a 'Security System' for our model.

Real-World Use Case

- **Scenario:** A Bank Loan Approval Model.
- **Input:** User Income.
- **Output:** Approved (1) or Rejected (0).
- **Change:** Economy booms (High Income) OR Bank gets stricter (Policy Change).
- **Risk:** Approving risky loans or rejecting good customers.

Input Data / Inputs

- **Synthetic Data Generation:**

1. **Baseline:** Normal operations (Year 2023).
2. **Batch 1:** 'Data Drift' - High Inflation (Year 2024).
3. **Batch 2:** 'Concept Drift' - New Strict Policy (Year 2025).

Concepts Used (High Level)

- **Data Quality:** Is the data broken? (Nulls, Errors)
- **Data Drift ($P(X)$):** Did the input data change?
- **Concept Drift ($P(Y|X)$):** Did the approval logic change?

Concepts Breakdown (Simple)

- **KS Test (Kolmogorov-Smirnov):**

Analogy: Comparing the shape of two heaps of sand.

Verdict: $p\text{-value} < 0.05$ means 'Shapes are DIFFERENT'.

- **Mean Shift:**

Analogy: Comparing the average height of students in two classes.

Verdict: $>20\%$ change means 'Shift Detected'.

Step-by-Step Solution Flow

- 1. **Ingest** new batch of data.
- 2. **Check 1:** Data Quality (Nulls? Range?). If Fail -> STOP.
- 3. **Check 2:** Drift (KS Test). If Fail -> ALERT.
- 4. **Alert:** Trigger email/slack to retraining team.

Code Logic Summary

- **Python Script:** drift_demo.py
- **Functions:**
 - generate_data(): Creates the 3 scenarios.
 - check_data_quality(): Validates health.
 - check_drift(): Runs statistical tests.
 - run_monitoring_pipeline(): Orchestrates the checks and prints alerts.

Important Functions & Parameters

- `stat, p_value = ks_2samp(data1, data2)`
 - `data1, data2`: The two arrays to compare.
 - **Returns:** `p_value` (Probability they are same).
- `pd.isnull().sum()`: Counts missing values.

Execution Output

- **Baseline Test:** ■ SYSTEM HEALTHY.
- **Batch 1 (Inflation):** ■ ALERT FIRED.
 - *Reason:* Significant distribution shift (Mean Income went up).
- **Batch 2 (Policy Change):** ■ SYSTEM HEALTHY (False Negative!).
 - *Observation:* Input distribution looked normal, but labels changed. Monitoring X alone missed this!

Observations & Insights

- **Drift Checks** are good at catching **Input Changes**.
- **Drift Checks** are BAD at catching **Logic Changes** (Concept Drift) unless we monitor predictions or targets.
- **Alerts** must be actionable ('Retrain', not just 'Error').

Advantages & Limitations

- **Pros:**

- Catches broken pipelines instantly (Nulls).
- Detects major environmental shifts (Inflation).

- **Cons:**

- Blind to Concept Drift without Ground Truth.
- KS Test can be too sensitive (Alert fatigue).

Interview Key Takeaways

- **Q:** Difference between Data & Concept Drift?
 - **A:** Data Drift = Input changes. Concept Drift = Logic/Relationship changes.
- **Q:** Which test for distribution check?
 - **A:** KS Test (Numerical) or Chi-Square (Categorical).

Conclusion

- Monitoring is NOT optional.
- We implemented a functioning pipeline with Data Quality and Drift Alerts.
- **Next Step:** Add 'Performance Monitoring' (Accuracy Check) to catch Concept Drift.